



# Btc4Stable Small

Security Review

17.05.2026

Conducted by :

**Synthrax**, Security Researcher



# Contents

|                                                                                                                                  |          |
|----------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>1 Disclaimer</b>                                                                                                              | <b>2</b> |
| <b>2 Protocol Overview</b>                                                                                                       | <b>2</b> |
| <b>3 Risk Classification</b>                                                                                                     | <b>2</b> |
| 3.1 Impact . . . . .                                                                                                             | 2        |
| 3.2 Likelihood . . . . .                                                                                                         | 2        |
| 3.3 Actions required by severity level . . . . .                                                                                 | 2        |
| <b>4 Findings Summary</b>                                                                                                        | <b>3</b> |
| <b>5 Detailed Findings</b>                                                                                                       | <b>4</b> |
| 5.1 Medium Findings . . . . .                                                                                                    | 4        |
| 5.1.1 [M-01] initializeMicroLoan(...) function has no slippage protection for collateral value . . . . .                         | 4        |
| 5.1.2 [M-02] TWAP resumes after a stale gap without rebuilding a clean observation window . . . . .                              | 4        |
| 5.2 Low Findings . . . . .                                                                                                       | 5        |
| 5.2.1 [L-01] updatePrice() can write depegged cbBtc price to priceObservations . . . . .                                         | 5        |
| 5.2.2 [L-02] lossBps calculation truncates and understates the stress in _applyBadDebtStress(...) . . . . .                      | 5        |
| 5.2.3 [L-03] emergencyWithdraw(uint256 amount) uses amount parameter for withdrawal, creating time-sensitive fragility . . . . . | 6        |
| 5.2.4 [L-04] _validateConfiguredSequencerFeed(...) accepts a down sequencer . . . . .                                            | 6        |
| 5.3 Informational Findings . . . . .                                                                                             | 6        |
| 5.3.1 [I-01] Full payments can emit partial payment event . . . . .                                                              | 6        |
| 5.3.2 [I-02] Users can withdraw funds during protocol pause . . . . .                                                            | 7        |
| 5.3.3 [I-03] previewMicroLoan(...) doesn't follow initializeMicroLoan(...) behaviour . . . . .                                   | 7        |
| 5.3.4 [I-04] updateAndFetchCbBtcPrice() returns cbBTC price paired with unrelated times-tamp . . . . .                           | 7        |

# 1 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where I try to find as many vulnerabilities as possible. I can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

# 2 Protocol Overview

BTC4Stable is a cbBTC-collateralized USDC microloan protocol deployed on Base (L2). cbBTC holders post Bitcoin collateral to take 30-day microloans ranging from 1 to 1,000 USDC, paying an upfront dynamic commitment fee. The protocol is non-liquidating — borrowers either repay within 30 days to reclaim their cbBTC, or forfeit it to the Treasury on default.

# 3 Risk Classification

| Severity           | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: High   | High         | High           | Medium      |
| Likelihood: Medium | High         | Medium         | Low         |
| Likelihood: Low    | Medium       | Low            | Low         |

## 3.1 Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - involves a small loss of funds or affects a core functionality of the protocol.
- **Low** - encompasses any unexpected behavior that is non-critical.

## 3.2 Likelihood

- **High** - a direct attack vector; the cost is relatively low compared to the potential loss of funds.
- **Medium** - only a conditionally incentivized attack vector, with a moderate likelihood.
- **Low** - involves too many or unlikely assumptions; offers little to no incentive.

## 3.3 Actions required by severity level

- **High** - client **must** fix the issue.
- **Medium** - client **should** fix the issue.
- **Low** - client **could** fix the issue.

## 4 Findings Summary

| ID     | Title                                                                                                                               | Severity      |
|--------|-------------------------------------------------------------------------------------------------------------------------------------|---------------|
| [M-01] | <code>initializeMicroLoan(...)</code> function has no slippage protection for collateral value                                      | Medium        |
| [M-02] | TWAP resumes after a stale gap without rebuilding a clean observation window                                                        | Medium        |
| [L-01] | <code>updatePrice()</code> can write depegged <code>cbBtc</code> price to <code>priceObservations</code>                            | Low           |
| [L-02] | <code>lossBps</code> calculation truncates and understates the stress in <code>_applyBadDebtStress(...)</code>                      | Low           |
| [L-03] | <code>emergencyWithdraw(uint256 amount)</code> uses <code>amount</code> parameter for withdrawal, creating time-sensitive fragility | Low           |
| [L-04] | <code>_validateConfiguredSequencerFeed(...)</code> accepts a down sequencer                                                         | Low           |
| [I-01] | Full payments can emit partial payment event                                                                                        | Informational |
| [I-02] | Users can withdraw funds during protocol pause                                                                                      | Informational |
| [I-03] | <code>previewMicroLoan(...)</code> doesn't follow <code>initializeMicroLoan(...)</code> behaviour                                   | Informational |
| [I-04] | <code>updateAndFetchCbBtcPrice()</code> returns <code>cbBTC</code> price paired with unrelated timestamp                            | Informational |

## 5 Detailed Findings

### 5.1 Medium Findings

#### 5.1.1 [M-01] `initializeMicroLoan(...)` function has no slippage protection for collateral value

**Severity:** Medium **Status:** Fixed

**File(s):** `Btc4Stable.sol`

**Description:** `initializeMicroLoan(...)` function checks for USDC fee slippage, but doesn't have any slippage protection for `cbBtc` collateral. If between `previewMicroLoan(...)` and `initializeMicroLoan(...)` calls the collateral amount needed changes (because of collateral token value drops or `setCollateralRatioBps(...)` call), users might pay more `cbBtc` than intended.

**Impact:** If `permit2` allowance is set more than necessary, users may pay more collateral and in case of immediate withdrawal will lose the fee paid for loan initialization.

**Recommendation(s):** Add a `maxCollateralAmount` parameter to `initializeMicroLoan(...)`, mirroring the existing `maxCommitmentFeeBps` guard, and `revert if quote.collateralAmount > maxCollateralAmount`.

#### 5.1.2 [M-02] TWAP resumes after a stale gap without rebuilding a clean observation window

**Severity:** Medium **Status:** Fixed

**File(s):** `TwapOracle.sol`

**Description:** `TwapOracle` can resume serving TWAPs immediately after a long observation gap even though the averaging window still contains stale price history. When `_writeObservation(int24 tick, uint32 blockTimestamp)` is called after a long interval without writes, it advances cumulative history with `tickCumulative += int56(lastTick) * int56(uint56(blockTimestamp - last.blockTimestamp))`, which models the entire gap as if the old tick remained valid until the new write.

The read path does not reject that bridged history. `_calculateTwap(uint32 secondsAgo)` checks only `_getOldestObservationTimestamp()` and then derives the average from `_observeSingle(currentTime, secondsAgo)` and `_observeSingle(currentTime, 0)`, without verifying that the requested window was rebuilt from recent, contiguous observations after the gap.

Because freshness and availability checks rely on the latest successful update and basic history coverage, a single post-gap write can make the oracle appear usable again even though the TWAP window is still contaminated by stale pre-gap data.

**Impact:** Oracle will return inaccurate TWAP values immediately after liveness resumes. Because `initializeMicroLoan(...)` gets the lowest of TWAP and spot values, users can get underestimated collateral values for loans if the price rose during the gap.

**Recommendation(s):** Rebuild the TWAP history after any observation gap that exceeds an acceptable staleness threshold.

## 5.2 Low Findings

### 5.2.1 [L-01] updatePrice() can write depegged cbBtc price to priceObservations

**Severity:** Low **Status:** Fixed

**File(s):** TwapOracle.sol

**Description:** updatePrice() has a check for the cbBtc depeg case:

```
if (state.cbBtcDepegged || state.cbBtcInStabilityPeriod) {  
    return;  
}
```

But the check happens before the `_fetchLatestPrice()` call, which is where the depegged state is going to be registered. This means that on the first depegged price update the call is going to execute successfully. Subsequent calls will be correctly blocked.

**Impact:** updatePrice() is going to write a depegged cbBtc price to priceObservations, which is later going to be used for further calculations.

**Recommendation(s):** Move the `state.cbBtcDepegged` check after the `_fetchLatestPrice()` call.

### 5.2.2 [L-02] lossBps calculation truncates and understates the stress in \_applyBadDebtStress(...)

**Severity:** Low **Status:** Fixed

**File(s):** Btc4Stable.sol

**Description:** \_applyBadDebtStress(...) first calculates the amount of bad debt represented in bps:

```
uint256 lossBps = totalLiquidityBefore == 0  
    ? cfg.maxBadDebtStressBps  
    : Math.mulDiv(lossUsdc, Constants.MAX_BPS, totalLiquidityBefore);
```

Because the calculation doesn't specify the rounding direction, the lossBps value is going to be truncated, and in case of sub 1 bps values it's going to return 0. Later, the truncated lossBps is used in the stressIncreaseSc calculation:

```
uint256 stressIncreaseSc =  
    Math.mulDiv(lossBps * Constants.FEE_MODEL_SCALE,  
        cfg.badDebtStressMultiplierBps,  
        Constants.MAX_BPS);
```

Multiplying the truncated lossBps worsens the situation further, returning an understated stressIncreaseSc value.

**Impact:** stressIncreaseSc will be constantly understated.

**Recommendation(s):** Incorporate FEE\_MODEL\_SCALE into the lossBps calculation before dividing by totalLiquidityBefore, and use ceiling rounding to prevent truncation from amplifying the error in stressIncreaseSc.

### 5.2.3 [L-03] emergencyWithdraw(uint256 amount) uses amount parameter for withdrawal, creating time-sensitive fragility

**Severity:** Low **Status:** Fixed

**File(s):** LendingPool.sol

**Description:** emergencyWithdraw(uint256 amount) relies on the amount parameter for the withdrawal amount instead of using balanceOf(). In case of an ongoing emergency, the amount of tokens held by the contract can change rapidly, and at the moment of the emergencyWithdraw(...) call the contract can contain fewer tokens than specified by the amount parameter, resulting in a time-sensitive call revert.

**Impact:** emergencyWithdraw(...) call can revert during an ongoing emergency.

**Recommendation(s):** Use balanceOf() instead of the amount parameter for withdrawals.

### 5.2.4 [L-04] \_validateConfiguredSequencerFeed(...) accepts a down sequencer

**Severity:** Low **Status:** Fixed

**File(s):** PriceOracle.sol

**Description:** The \_validateConfiguredSequencerFeed(...) validation condition if ((answer != 0 && answer != 1) || updatedAt < startedAt) passes successfully when answer == 1, which indicates that the sequencer is offline per Chainlink semantics. This can result in the registration of a nonfunctional sequencer.

**Impact:** At runtime every oracle call is gated by \_checkSequencer() which correctly requires answer == 0, meaning all loan originations immediately revert until the sequencer recovers.

**Recommendation(s):** Require that \_validateConfiguredSequencerFeed(...) accepts only active sequencers.

## 5.3 Informational Findings

### 5.3.1 [I-01] Full payments can emit partial payment event

**Severity:** Informational **Status:** Fixed

**File(s):** Btc4Stable.sol

**Description:** makeUsdcPayment(...) function emits an event on partial payments:

```
if (isPartial) {
    emit USDCPartialPaymentReceived(loanId, actualPaymentAmount, remainingBalance);
}
```

If the isPartialUsdcPayment parameter is set to true but partialPaymentAmount covers the whole debt, the function will still emit the USDCPartialPaymentReceived event.

**Impact:** Full payments can emit USDCPartialPaymentReceived event, causing off-chain monitoring system confusion.

**Recommendation(s):** Before emitting USDCPartialPaymentReceived, check that remainingBalance > 0.

### 5.3.2 [I-02] Users can withdraw funds during protocol pause

**Severity:** Informational **Status:** Fixed

**File(s):** Btc4Stable.sol

**Description:** pause() call affects only new loan generations. claimCollateral() function is still callable during protocol pause. This can put protocol-held funds at risk. The protocol architecture allows user claims to be paused during emergency events without putting any funds at risk, because the protocol has no liquidation mechanism and loans can still be repaid during the pause through the makeUsdcPayment(...) function call to meet the repayment due time.

**Impact:** During emergency events, no funds should be allowed to be withdrawn by users until admins resolve all issues.

**Recommendation(s):** Disallow claimCollateral() calls during protocol pause.

### 5.3.3 [I-03] previewMicroLoan(...) doesn't follow initializeMicroLoan(...) behaviour

**Severity:** Informational **Status:** Fixed

**File(s):** Btc4Stable.sol

**Description:** previewMicroLoan(...) function doesn't mirror initializeMicroLoan(...) error patterns. Specifically, it doesn't throw an error during cbBtc depeg events and in case of USDC shortage for the requested loan amount.

**Impact:** previewMicroLoan(...) will return a valid loan quote which is actually unexecutable. Impact is limited to view/preview paths only.

**Recommendation(s):** Add cbBtc depeg checks to getValidatedCbBtcSpotPrice() and add \_validateLoanPrerequisites checking to previewMicroLoan(...), so it mirrors initializeMicroLoan(...) behaviour.

### 5.3.4 [I-04] updateAndFetchCbBtcPrice() returns cbBTC price paired with unrelated timestamp

**Severity:** Informational **Status:** Fixed

**File(s):** PriceOracle.sol

**Description:** updateAndFetchCbBtcPrice() function returns (cbBtcPrice, Math.min(cbBtcUpdatedAt, btcUpdatedAt)) — the returned price is always the cbBTC/USD value, but the accompanying updatedAt may be the BTC/USD feed's (older) timestamp, so any caller reasoning about the freshness of the returned price may receive the age of a different asset.

**Impact:** Currently the protocol only uses this timestamp for a future-timestamp guard (> block.timestamp) so there is no active exploit path, but future callers face a semantic trap.

**Recommendation(s):** Return the corresponding updatedAt value paired with the cbBtcPrice price.